

软件系统设计 设计模式

一、 软件设计引入

需求定义系统需要满足的目标（用户需求指出了目标，比如在线会议的目标是能够看到人员、听到声音等）。

规约定义系统的外部可观察行为。

架构定义系统级的主要组成部分、各部分之间的互动方式、使用的技术（比如在线会议需要前端、操作系统调用功能部分、网络部分以及部分之间的联系，使用什么技术等等）。

设计定义如何完成任务、编写代码，我们关注面向对象设计。

二、 面向对象设计

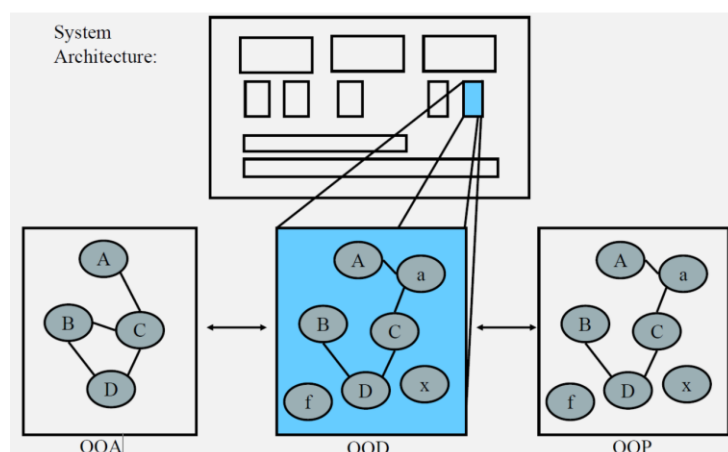
（一）什么是面向对象设计 OOD

面向对象对象设计是将实现的约束条件（性能约束、应对变化的需求等）应用到面向对象分析所产生的概念模型（对应问题域，如数据库、文件、用户界面、算法等）的过程（问题驱动）。

（二）系统设计步骤

1. 用方法和属性来描述用于构成系统的类，得到相应的概念模型。
2. 在概念模型的基础上，添加明显不属于领域的类，比如抽象类和接口。
3. 描述类是如何构成组件的。

（三）OOA、OOD 和 OOP



针对已有的系统架构（包含不同组件），针对问题进行面向对象分析，得到 OOA 的概念模型，然后进行 OOD 的面向对象设计添加明显不属于领域的类，最后我们将 OOD 得到的结果映射到 OOP 中。

（四）面向对象设计的困难

面向对象设计的困难是**将系统分解成对象**。对象可能来自于分析模型、实现空间（数据库、文件、UI、IPC）等，当然也可能其他类没有这样的类（使得特殊的设计更通用，比如策略模式）。

从 OOA 到 OOD 没有循序渐进的简单方式，虽然 OOA 以直接的方式给出了问题域组件，但是还需要经验才能完成从 OOA 到 OOD 的转变。

（五）软件系统设计经验

经验丰富的设计师可以看到同样的老问题反复出现，每次遇到类似的问题，设计师都会从行而有效的东西（设计原则以及经典的设计经验）入手。

三、面向对象设计原则

（一）面向对象设计原则

可维护性低的软件设计的原因：**过于僵硬（硬编码）、过于脆弱（修改导致的未知影响）、复用率低（不可黑盒使用）、粘度过高（架构需要修改）**

好的系统设计应具备的性质：**可扩展性、灵活性和可插入性**。

（二）软件的可维护性和可复用性

软件的**复用或重用**优点：提高软件的开发效率，提高软件质量，节约开发成本

1. 合适复用可以改善系统的可维护性。
2. 部分复用可能破坏系统的可维护性，比如 A 和 B 修改 C，如果 A 需要 C 多提供一个行为，而 B 不需要则会有问题。

面向对象设计复用的目标在于**实现支持可维护性的复用**。

在面向对象设计中，可维护性复用都是以**面向对象设计原则**为基础的。

面向对象设计原则也是对系统进行合理**重构**的指南针。

重构是在不改变软件现有功能的基础上，通过调整程序代码改善软件质量、性能，使其程序的设计模式和架构更趋合理，提高软件的扩展性和维护性。

(三) 面向对象设计原则

设计原则名称	设计原则简介	设计原则之间关系
单一职责原则 SRP Single Responsibility Principle	一个对象应该只包含 单一的职责 ，并且该职责被完整地封装在一个类中。	
开闭原则 OCP Open-Closed Principle	一个软件实体应该 对扩展开放，对修改关闭 。软件实体可以是一个软件模块、一个由多个类组成的局部结构或一个单独的类。	开闭原则是对单一职责原则的加强。
里氏代换原则 LSP Liskov Substitution Principle	所有引用基类（父类）的地方必须能 透明地 使用其子类的对象。 通俗表达 ：软件中如果能够使用基类对象，那么一定能够使用其子类对象。	里氏代换原则是开闭原则的具体实现。
依赖倒转原则 DIP Dependency Inversion Principle	高层模块 不应该依赖于低层模块 ，他们都应该依赖抽象。抽象不应该依赖于细节，细节应该依赖于抽象。 另一种表述 ：要针对接口编程，而不要针对实现编程。	依赖倒转原则是开闭原则的具体实现。
接口隔离原则 ISP Interface Segregation Principle	客户 不应该依赖 那些它不需要的接口。	拆分时需要满足单一职责原则
合成复用原则 CRP Composite Reuse Principle	在系统中应该尽量多实用组合聚合关联关系，尽量少甚至不使用继承关系。	
迪米特法则 LoD Law of Demter	在一个软件实体对其他实体的引用越少越好，或者说如果两个类不必彼此直接通信，那么这两个类就不应当发生直接的相互作用，而是通过引入一个第三者发生简介交互。	

1. 单一职责原则：实现**高内聚、低耦合**的指导方针

一个类（大到模块，小到方法）承担的责任越多，被复用的可能性就越小。类的职责包括数据职责（属性体现）和行为职责（方法体现）两个方面。

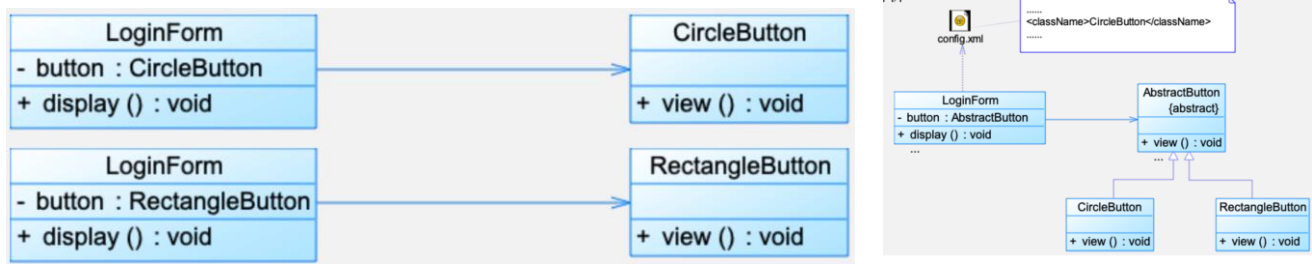
单一职责原则实例：登录功能的实现，将 Login 类拆分成右图的三部分，将职责进行分离。



2. 开闭原则：关键是抽象化

设计模块时，应当保证该模块可以在不被修改的前提下被扩展。还可以通过对可变性封装原则（Principle of Encapsulation of Variation, EVP）来描述。

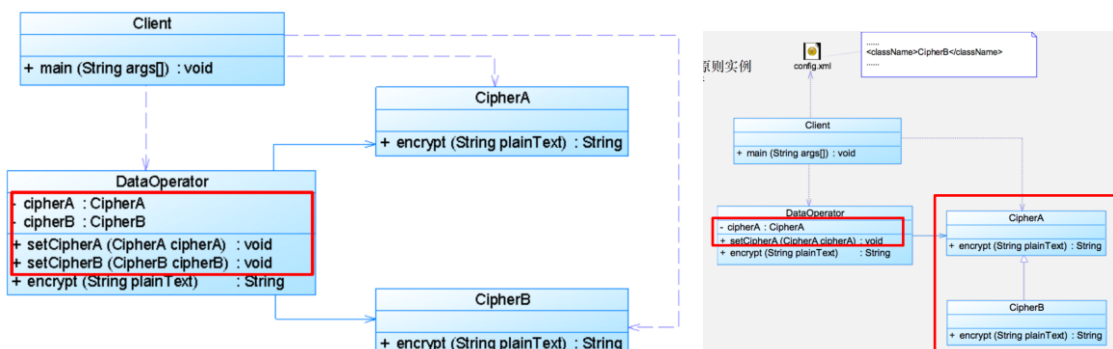
开闭原则实例：图形界面下不同形状的按钮，修改为右侧，将代码变为配置 XML 文件，结合反射完成。



3. 里氏代换原则

严格定义：如果对每一个类型为 S 的对象 o1，都有类型为 T 的对象 o2，使得以 T 定义的所有程序 P 在所有对象 o2 都替换成 o1 时，程序 P 的行为没有变化，那么类型 S 是类型 T 的子类型。子类不应该是父类的功能的扩展，我们可以使用组合的方式来扩展功能。

里氏代换原则实例：数据加密，为 CipherA 和 CipherB 设计共同父类，CipherB 在 CipherA 的基础上加密



4. 依赖倒转原则：关键是抽象化，并从抽象中导出具体实现

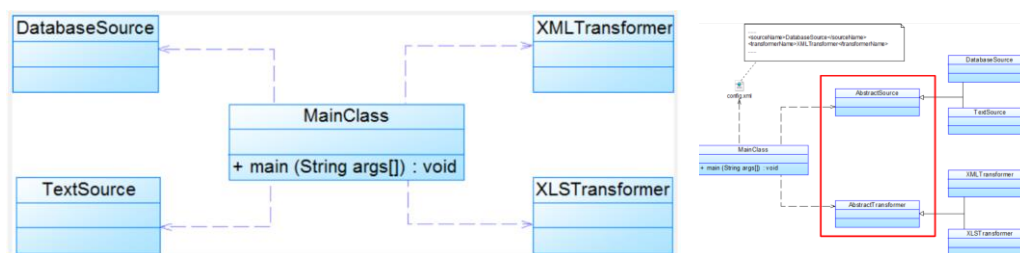
常见的实现方式：在代码中使用抽象类，而将具体类放在配置文件（将抽象放进代码，将细节放进元数据）

类之间的耦合：包括零耦合（最好）、具体耦合和抽象耦合关系（依赖倒转要求至少有一端是抽象的）。

依赖注入

1. 构造注入：通过构造函数注入实例变量
2. 设值注入：通过 **Setter** 方法注入实例变量
3. 接口注入：通过接口方法注入实例变量

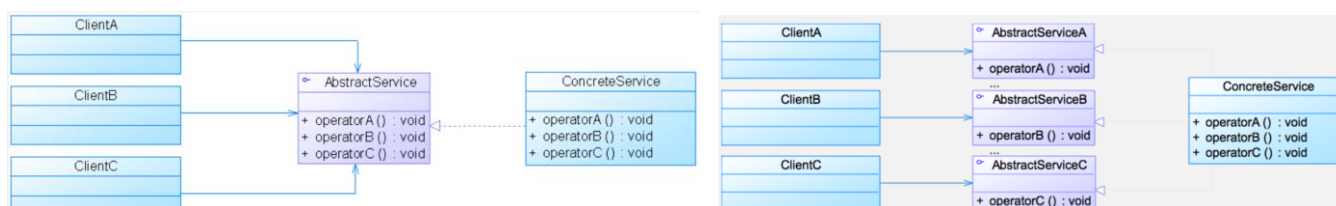
依赖倒转原则实例：数据转换面临增加新的数据源和文件格式的需求。



5. 接口隔离原则

接口隔离原则使用多个专门的接口（仅提供客户端需要的行为），而不使用单一的总接口，每个接口都应该承担一种相对独立的角色。可以在系统设计时采用定制服务的放肆，为不同的客户端提供宽窄不同的接口。

接口隔离原则实例：从左侧到右侧进行接口拆分。



6. 合成复用原则

聚合与组合区别：聚合（空心菱形箭头）是独立的，组合（实心菱形箭头）局部往往不能离开总体。

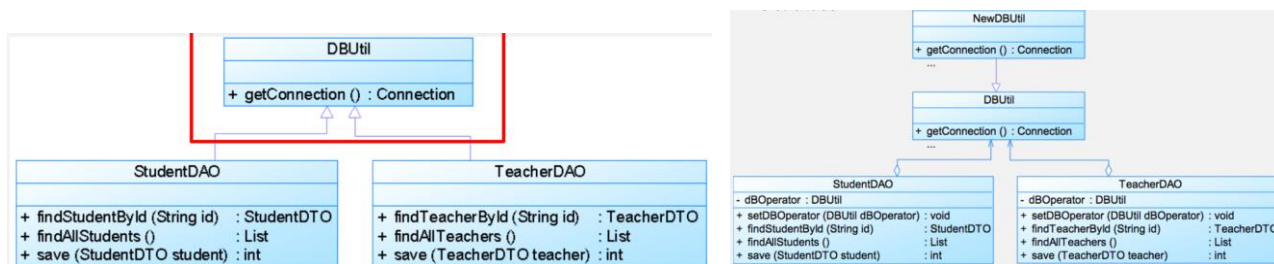
具体来讲，合成复用原则是在一个对象中通过关联关系（聚合/组合）来使用一些已有对象，使之成为新对象的一部分；新对象通过委派调用已有对象的方法达到复用其已有功能的目的。

复用方式

1. 继承复用（“白箱”复用）：实现简单，易于扩展，破坏系统封装性，静态实现灵活性不足，适用环境有限。

2. 组合/聚合复用（“黑箱”复用）：耦合性低，选择性调用成员对象较为灵活。

合成复用原则实例：教学管理系统的数据库访问。对于左侧，如果数据库连接方式变化，采用原本的 JDBC 方式连接数据库则需要修改 DBUtil 类源代码。同时如果 StudentDAO 和 TeacherDAO 如果数据库访问方式不同则需要添加新的 DBUtil 类并修改 StudentDAO 和 TeacherDAO 的源代码，违背开闭原则。



7. 迪米特法则：一个软件实体尽可能少的与其他实体发生相互作用

典型定义

1. 不要与陌生人说话
2. 只与你的直接朋友通信
3. 每一个软件单位对其他的单位都只有最少的知识，而且局限于那些与本单位密切相关的软件单位。

对一个对象的朋友类别（否则为陌生人）

1. 当前对象本身
2. 以参数形式传入到当前对象方法中的对象
3. 当前对象的成员变量
4. 如果当前对象的成员对象是一个集合，那么集合中的元素也都是朋友
5. 当前对象所创建的对象。

迪米特法则分类

1. 狭义迪米特法则

- a) 如果两个类之间不必彼此直接通信，那么这两个类就不应当发生直接的相互作用，如果需要方法调用，则可以通过**第三者**来转发这个调用（A 调用第三者 B，B 调用 C 中的方法）。
- b) 降低类之间的耦合，但是会导致大量小方法散落在系统中的各个角落从而导致系统不同模块之间的通信效率降低。

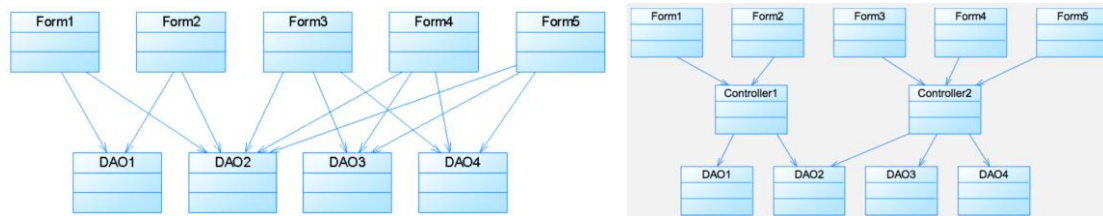
2. 广义迪米特法则

- a) 指对对象之间的信息流量、流向以及信息的影响的控制，主要是对信息隐藏的控制。
- b) 信息隐藏使得各个子系统之间**脱耦**，从而允许它们独立地被开发、优化、使用和修改，同时促进软件的**复用**。系统规模越大，信息隐藏就越重要。

迪米特法则的实现

1. 类的划分上：创建松耦合的类，提高类复用的可能性。
2. 类的结构设计上：每一个类都尽量降低其成员变量和成员函数的访问权限。
3. 类的设计上：如果可能一个类型应该被设计为不变类。
4. 其他类的引用上：一个对象对其他对象的引用应当降低到最低。

迪米特法则的实例：从左侧变为右侧



四、 具体设计模式

模式一般都有的缺点

1. 增加设计的复杂性和增加类的个数(增加辅助类)
2. 增加隔阂、方法调用，降低软件运行的效率，但是已经不是目前主要的问题

引入设计模式的作用

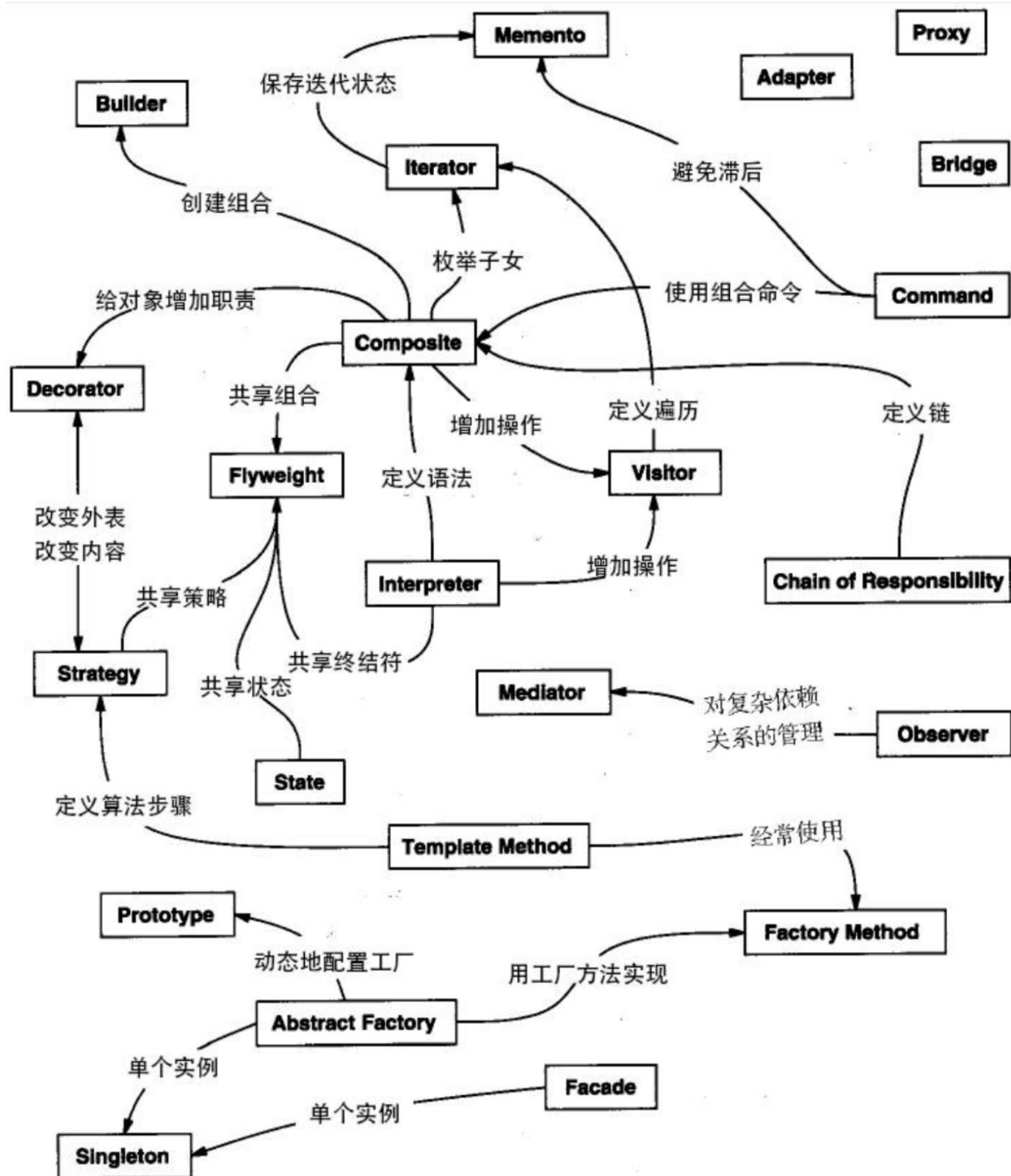
1. 设计模式提供了与其他开发人员**共享的词汇表**。
 - a) 使用模式和其他开发人员沟通时不仅仅传递了模式名称，还包含整套质量属性、特征和约束。
 - b) 模式可以让你用更少的话表达更多，帮助其他开发人员快速准确地了解您所考虑的设计。
2. 通过在模式级别（而不是实质性对象级别）进行思考，提高对体系结构的思考（但不要迷失在细节中）

为什么不建立设计模式的库：因为设计模式比库更高级别，设计模式告诉我们如何构造类和对象以解决某些问题。

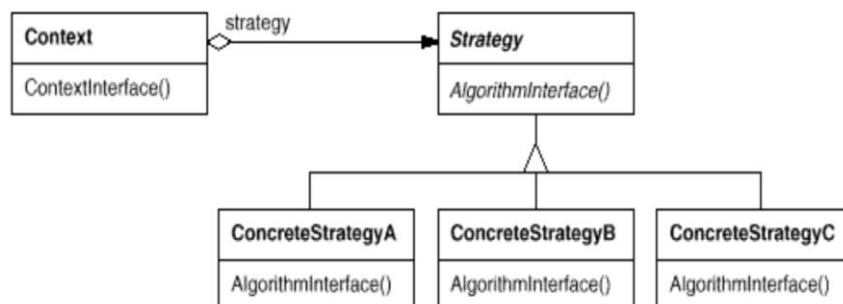
库和框架也是设计模式？库和框架不是设计模式，他们提供了具体到代码的特定实现，但是库和框架会使用设计模式。

面向对象的基础：抽象、封装、多态性、继承

设计模式分类：从模式本身：创建型、结构型、行为型，从模式实现：类模式（以继承为主要方式实现）、对象模式（以合成为主要方式实现）



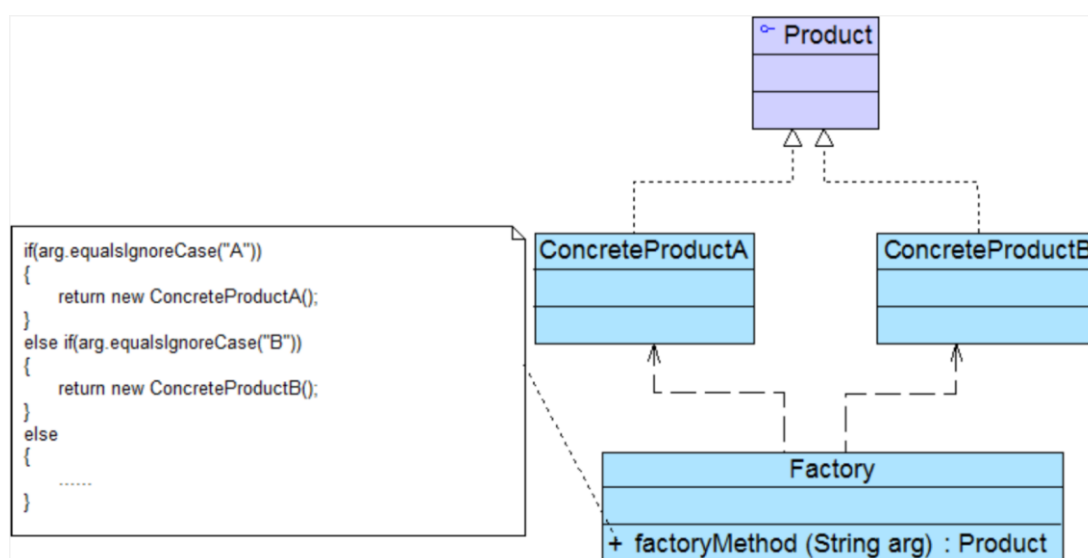
1. 策略模式



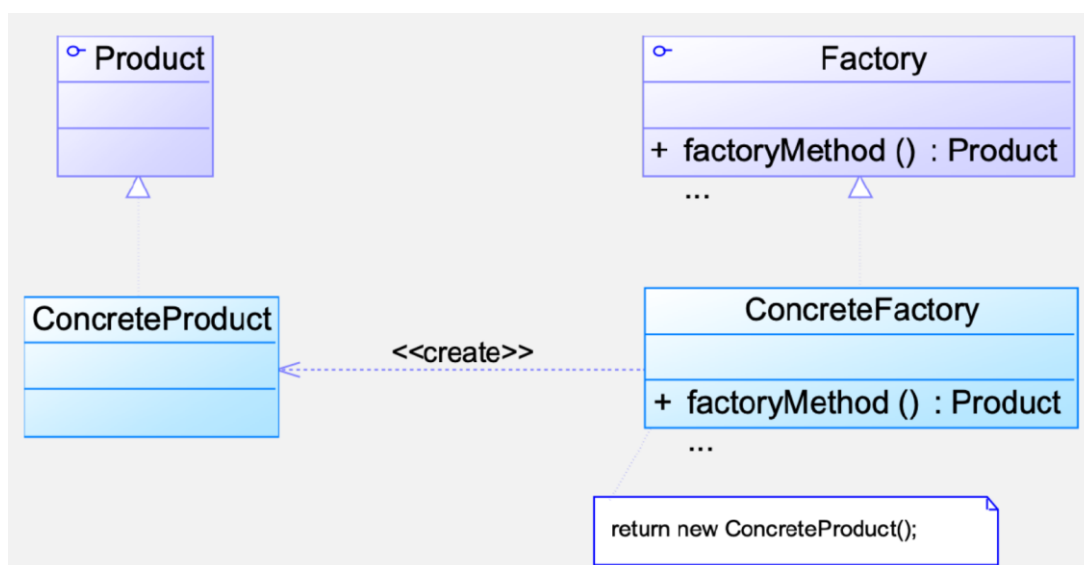
策略类的层次结构定义了一系列算法或行为，以供上下文重用。

修改策略不符合开闭原则，增加策略符合开闭原则。

2. 简单工厂模式



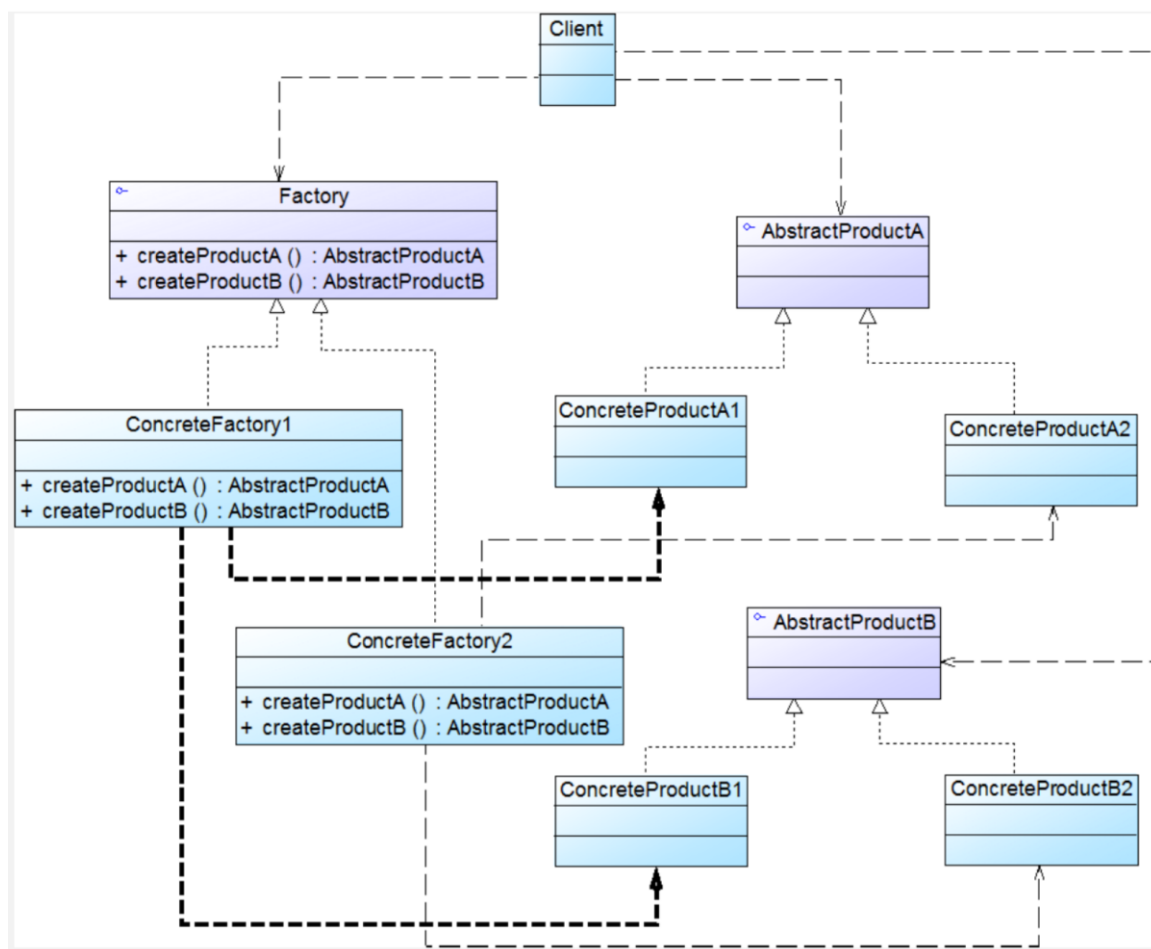
3. 工厂方法模式



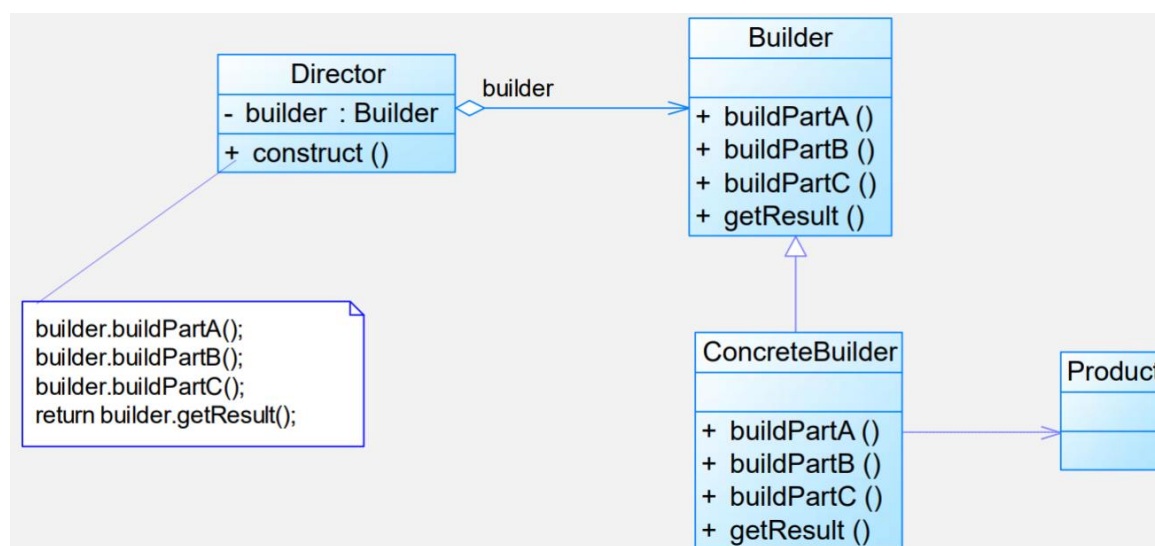
4. 抽象工厂模式

产品等级结构：产品等级结构即产品的继承结构。比如，抽象类是电视机，子类有海尔电视机等。

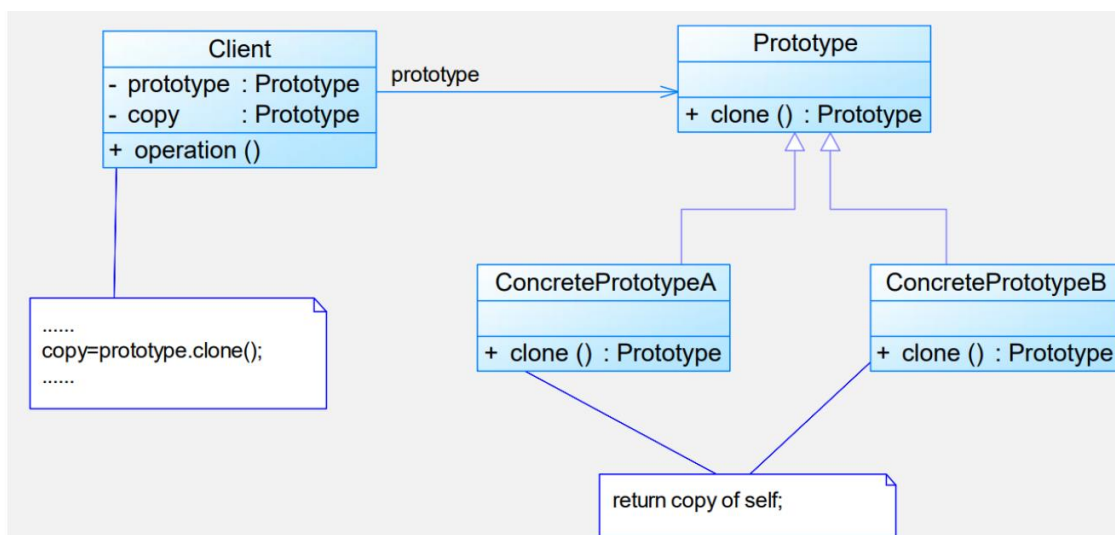
产品族：由同一个工厂生成的，位于不同产品等级结构中的一组产品。比如，海尔电视机在电视机的产品族中。



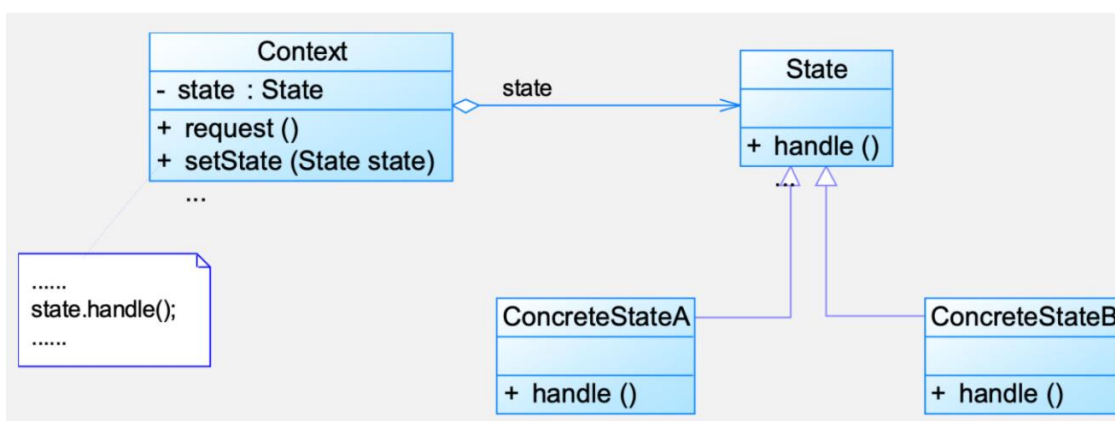
5. 建造者模式



6. 原型模式

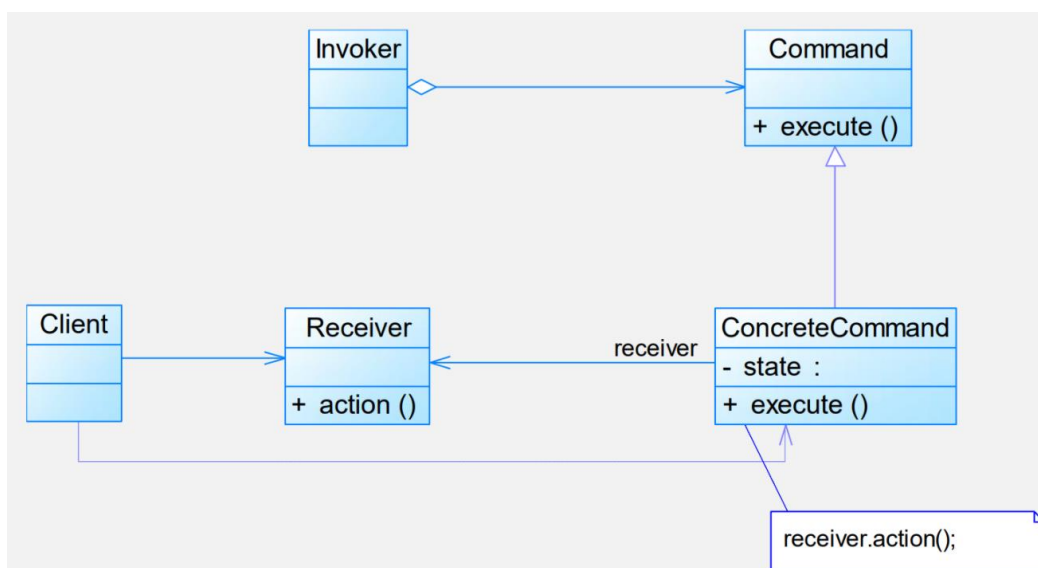


7. 状态模式

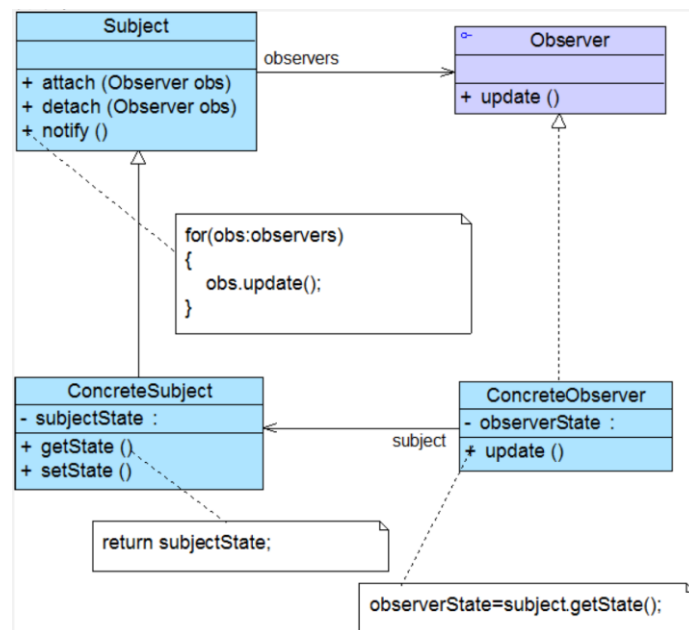


在结构上与策略模式是一致的，但是在使用上是很不同的，Context 是状态模式关联的上下文环境。

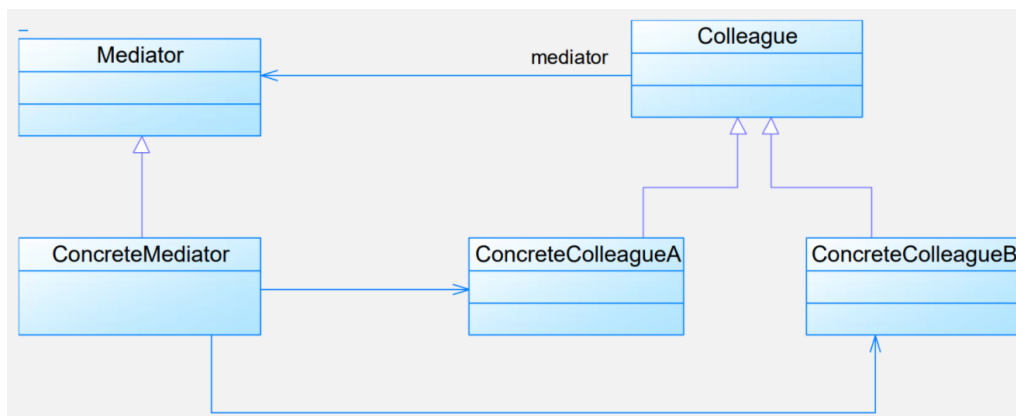
8. 命令模式



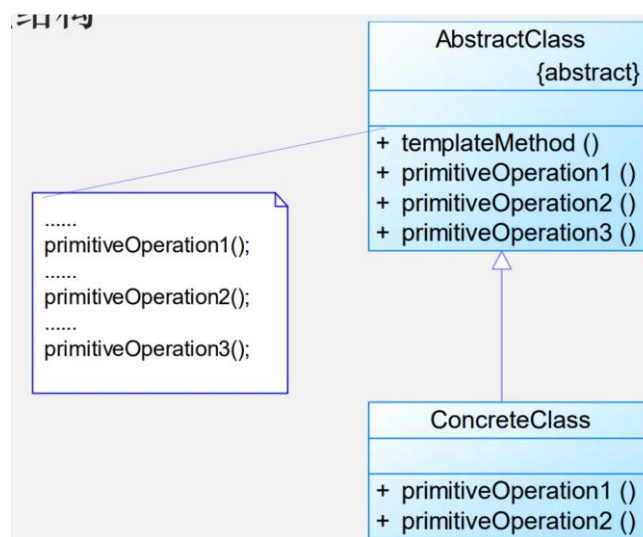
9. 观察者模式



10. 中介者模式



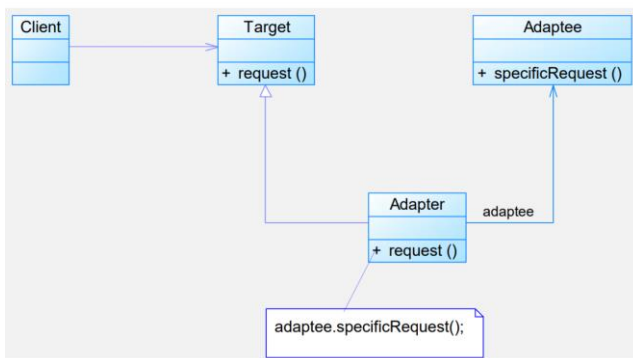
11. 模板方法模式



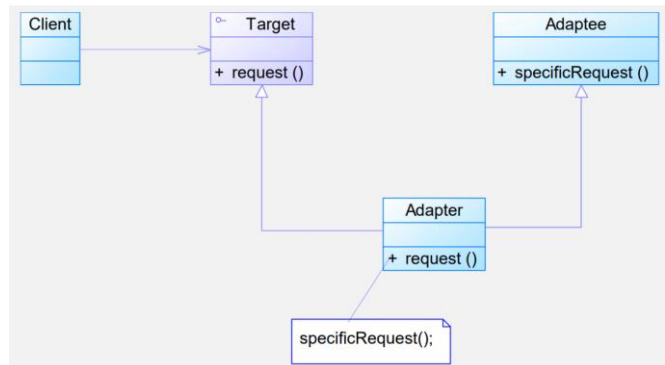
因为是类行为型模式，所以其结构图中只有类之间的继承关系，没有对象关联关系。

12. 适配器模式

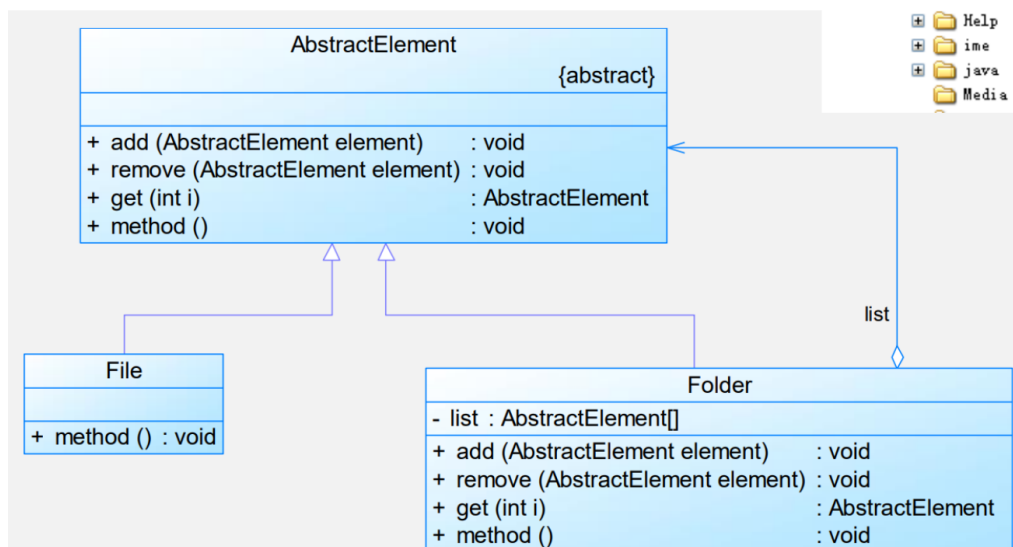
1. 对象适配器



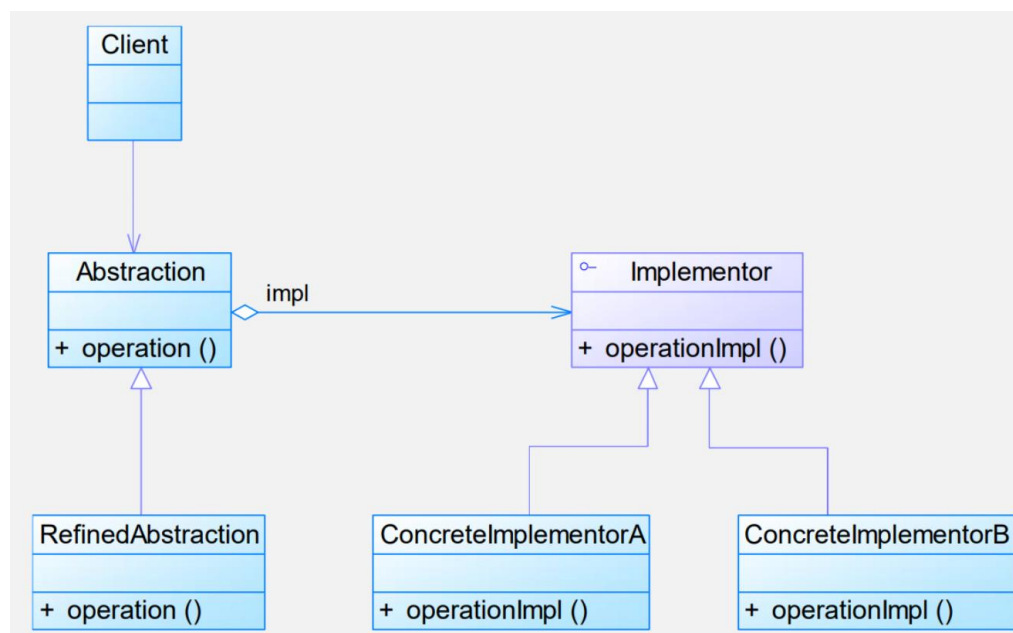
2. 类适配器



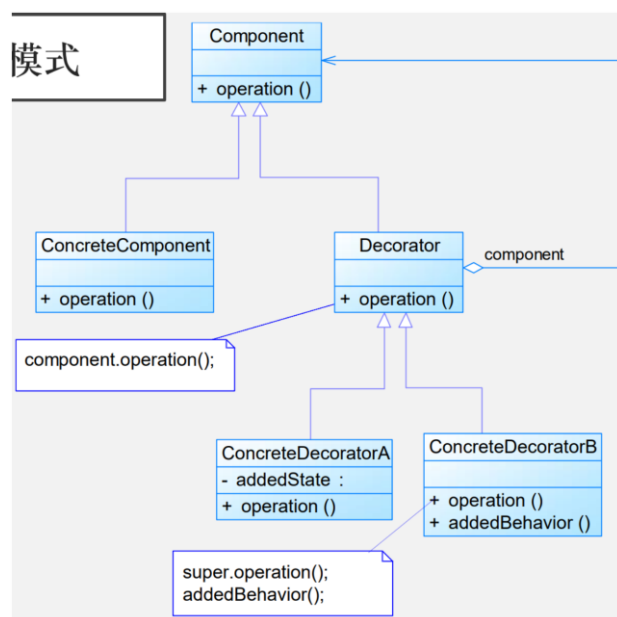
13. 组合模式



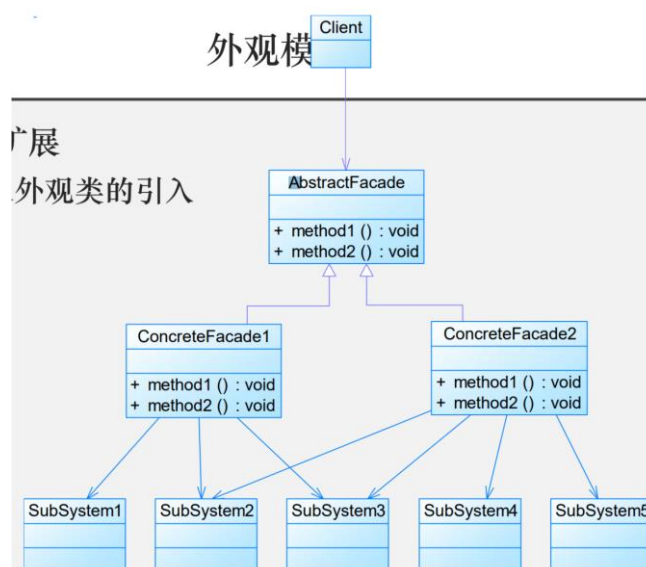
14. 桥接模式



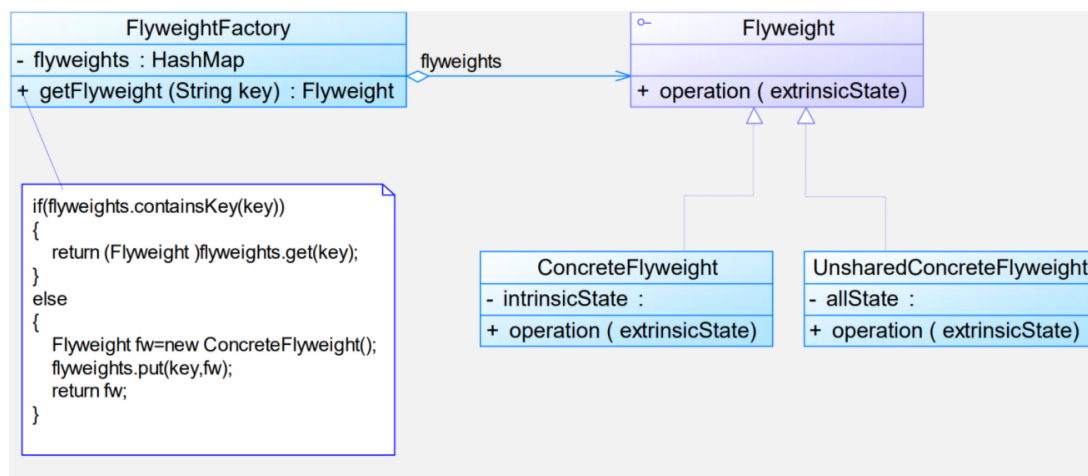
15. 装饰者模式



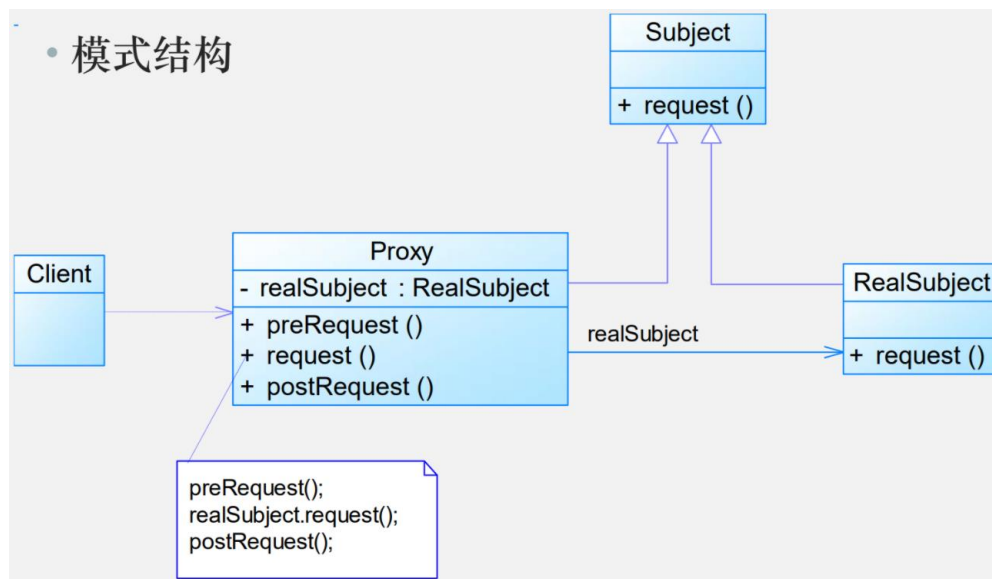
16. 外观模式



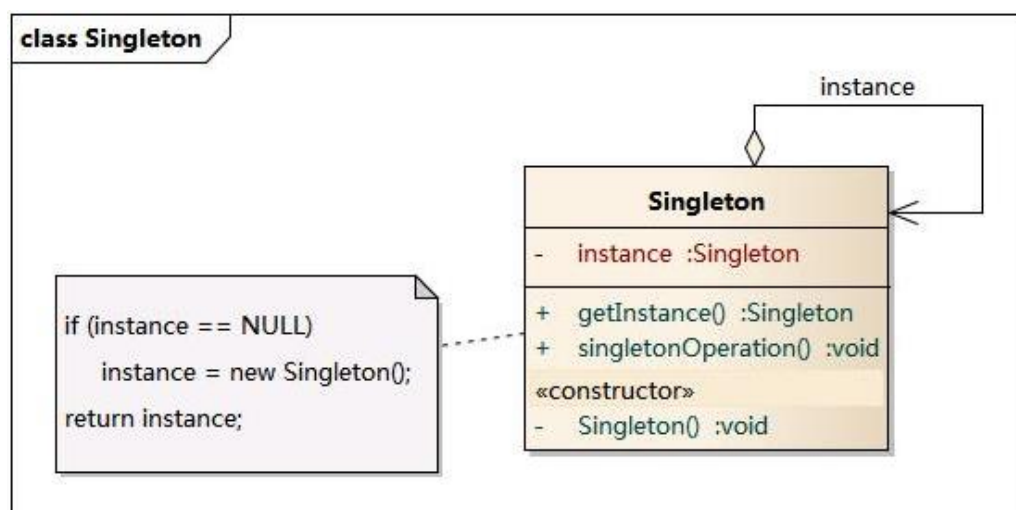
17. 享元模式



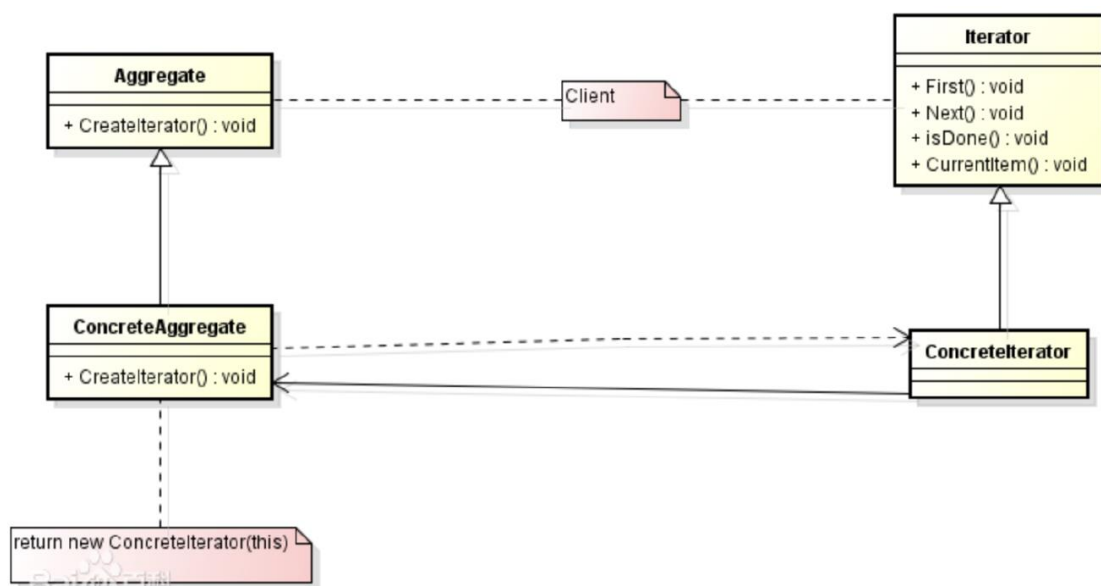
18.代理模式



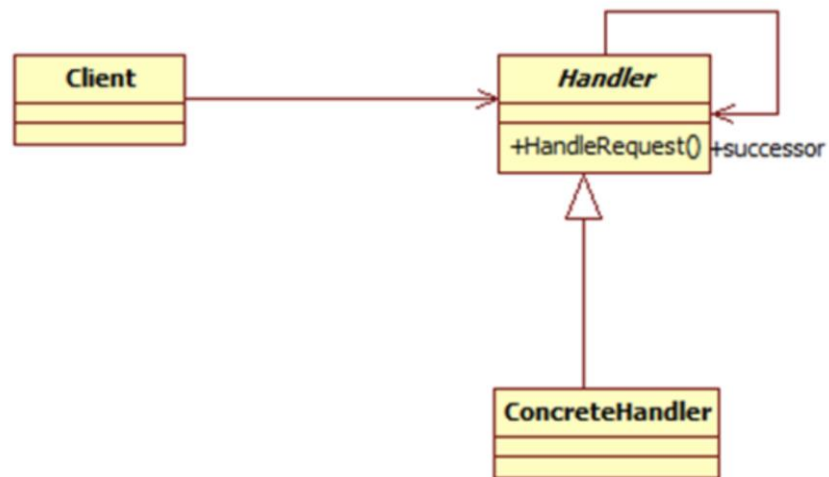
19.单例模式



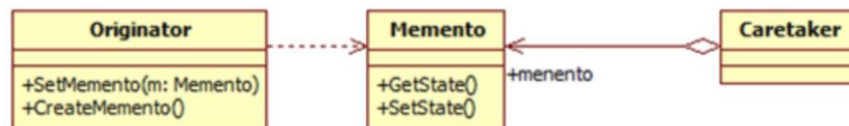
20.迭代器模式



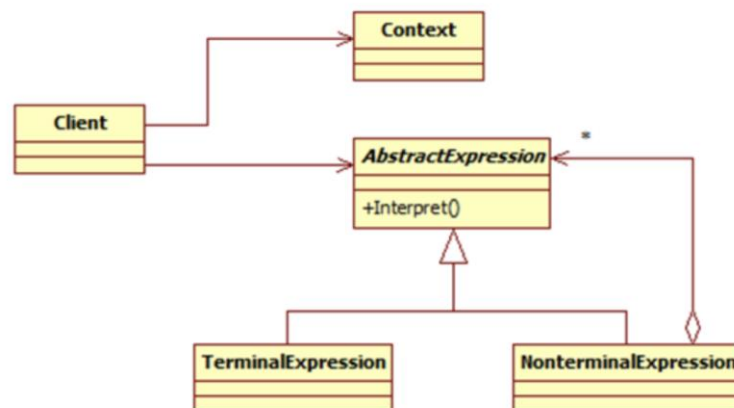
21. 责任链模式



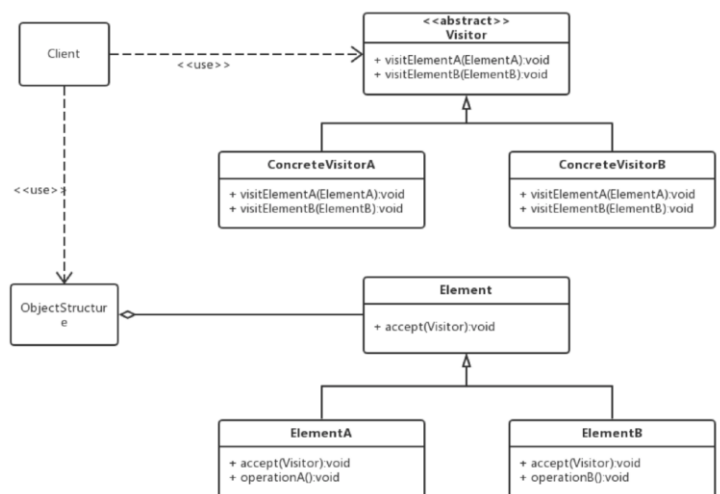
22. 备忘录模式



23. 解释器模式



24. 访问者模式



五、 防御式编程

(一) 什么是防御式编程

可以预见到(或至少预先推测到)问题所在,断定代码中每个阶段可能出现的错误,并作出相应的防范措施,来防止类似意外的发生。

(二) 防御式编程技巧

1. 使用良好的编码风格和合理的设计
2. 不要仓促地编写代码
3. 不要相信任何人
4. 编写清晰而非简洁的代码
5. 不让其他人做不该做的修补工作
6. 编译时打开所有警告开关
7. 检查所有的返回值
8. 在声明位置初始化所有变量
9. 尽可能推迟一些变量声明
10. 使用安全的数据结构
11. 使用标准语言工具
12. 审慎地进行强制转换

(三) 断言

1. 什么是断言

在开发期间使用的、让程序在运行时进行自检的代码,是对开发人员的警告,通常是一个子程序或宏。断言不可以有副作用。

2. 断言用途

输入输出参数取值处于预期范围

子程序开始（结束）时，文件流处于打开（关闭）状态

子程序开始（结束）时，文件流处于开头（结尾）位置

文件或流已用只读、只写或可读可写方式打开

仅用于输入的变量值没有被子程序修改

指针非空

传入子程序的数组或其他容器至少能容纳一定数量的元素

表已经初始化且保存了真实的数值

子程序开始（结束）执行时，某个容器是空的（满的）

一个经过高度优化的复杂子程序的运算结果和相对缓慢但代码清晰的子程序运算结果的一致性

3. 什么时候使用断言

断言主要用于**开发和维护**阶段，帮助查清互相矛盾的假定、预料之外的情况、传递给子程序的错误数据，生成产品代码时不编译进去。

4. 约束

契约式编程的一种，包含：

PreConditions(前置)：不要断言方法的规范、可以断言非 **public** 方法的前置条件

PostConditions(后置)

Invariants(某一个执行点不变式)

内部不变式：运行到特定时刻为真

控制流不变式：断言不可能被运行到的代码

类不变式：类对象作为有效的类成员必须满足的条件

5. 断言示例

冒泡排序的前置条件为不为空，后置条件为有序。

(四) 错误处理技术

1. 什么是错误

错误包括用户错误（输入错误）、程序错误（Bug）、意外情况（网络连接失败）。

2. 可能的错误处理技术：根据软件类别平衡正确性和健壮性（哪个优先级更高）

返回中立值：0、空串、空指针

换用下一个正确的数据

返回与前次相同的数据：windows 重启

换用最接近的合法值

将错误警告记录到日志文件中

返回一个错误码，用语言内建的异常机制抛出一个异常

调用错误处理子程序或对象：全局的错误处理子程序或对象

显示出错信息：不要告诉过多

用最妥当的方法在局部处理错误

关闭程序

3. 错误处理代码示例

1

```
void flowErrorHandling()
{
    if (operationOne())
    {
        if (operationTwo())
        {
            if (operationThree())
            { ... do more ... }
        }
    }
}
```

2

```
void flattenedErrorHandling()
{
    bool ok = operationOne();
    if (ok)
        ok = operationTwo();
    if (ok)
        ok = operationThree()
    if (ok)
        .. do more ...
    if (!ok)
    {
        ...clean up after errors ...
    }
}
```

3

```
void shortCircuitErrorHandling()
{
    if (!operationOne()) return;
    ... do something ...
    if (!operationTwo()) return;
    ... do something ..
    if (!operationThree()) return;
    ... do something ...
}
```

4

```
void gotoHell()
{
    if (!operationOne()) goto error;
    ... do something ...
    if (!operationTwo()) goto error;
    ... do something ...
    if (!operationThree()) goto error;
    ... do something ...
    return;
error:
    ... clean up after errors ...
}
```

5

```
void ExceptionalHandling()
{
    try {
        operationOne();
        operationTwo();
        operationThree();
    }
    catch( ... )
    {
        .. Cleanup after erros ...
    }
}
```

1 号 workflow 可读性差，2 号扁平化需要明显的控制变量，3 号短路需要做之后的处

理，4号合适的，5号异常，异常处理不明显。

4. 错误信息编写

- 以用户期望可以理解的方式呈现信息
- 不要显示没有意义的错误代码
- 将后果严重的错误与仅仅是警告区分开
- 提示用户每种选择可能的后果再提出问题

(五) 异常

1. 什么是异常

是将代码中的错误或异常事件传递给调用方代码的一种特殊手段，谨慎使用可以降低复杂度。

2. 不同语言的异常处理

表8-1 支持几种流行的编程语言的表达式			
跟异常相关的特性	C++	Java	Visual Basic
支持 try-catch 语句	支持		
支持 try-catch-finally 语句	不支持		
能抛出什么	std::exception 对象或 std::exception 派生类的对象；对象指针；对象引用；string 或 int 等数据类型	Exception 对象或 Exception 派生类的对象	Exception 对象或 Exception 派生类的对象
未捕获的异常所造成的影响	调用 std::unexpected() 函数，该函数在默认情况下将调用 std::terminate()，而这一函数在默认情况下又将调用 abort()	如果是一个“受检异常 (checked exception)”则终止正在执行的线程；如果是“运行时异常(runtime exception)”则不产生任何影响	终止程序执行
必须在类的接口中定义可能会抛出的异常	否	是	否
必须在类的接口中定义可能会捕获的异常	否	是	否

Q: 为什么C++没有 finally ?

为什么 C++ 没有 **finally** 呢？因为 C++ 提供了资源获得和初始化的 **RAI**，确保无论是否发生异常都可以完成清理工作（析构函数）

3. 异常使用建议

异常在真正例外（无法解决）时，通知程序其他部分，发生了不可忽略的错误。

不可以使用异常推卸责任，可以在局部处理就应该在局部处理。

避免在构造函数和析构函数中抛出异常：会导致资源泄露。

在适当的抽象层次抛出异常：保证异常的抽象层次与子程序接口的是一致的。

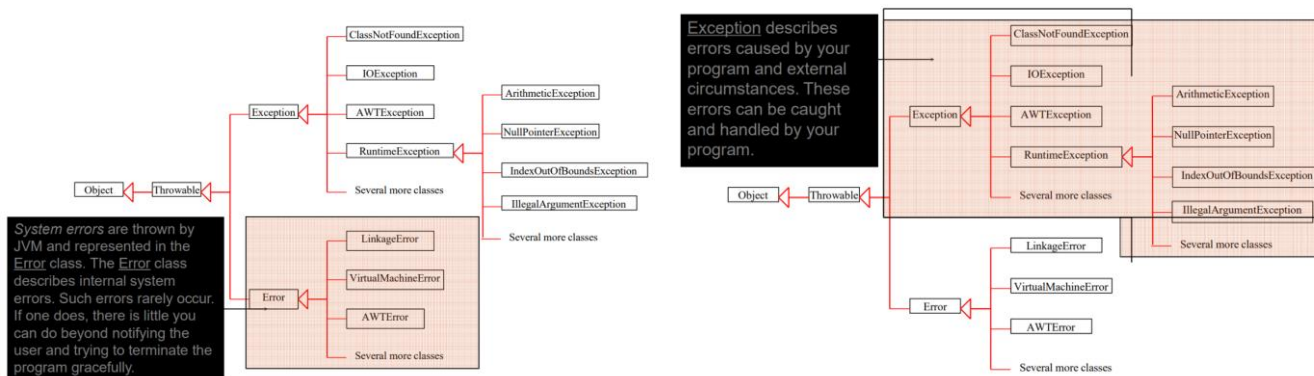
在异常消息中加入导致异常发生的全部信息：比如数组下标错误。

了解所用函数库可能抛出的异常。

将项目中对异常的使用标准化：建立自己的异常体系、确定是否使用集中的异常报告机制。

考虑是否真的需要处理异常。

4. Java 异常体系



（六）隔栏：包容错误

1. 什么是隔栏

隔栏是以防御式编程为目的而进行隔离的一种方法，将某些接口选定为“安全”区域的边界，对穿越安全区域边界的数据进行合法性检验（集中工作在特定的模块中降成本）。

2. 隔栏在做什么

在输入数据时将其转换为恰当的类型。

隔栏的部分包含了脏数据，隔栏外部程序应当使用错误处理技术，通过隔栏的数据之后的是干净数据。

（七）进攻式编程

1. 什么是进攻式编程

主动暴露出可能出现的错误，在开发阶段将其暴露显现出来，而在产品代码运行时让他能够自我恢复。

2. 进攻式编程方法

确保断言语使程序终止运行

完全填充所分配的所有内存

完全填充所分配到的所有文件或流

确保每个 **case** 语句的分支都可以产生严重错误

在删除一个对象之前把它填满垃圾数据

让程序将错误日志文件通过电子邮件发送

（八）辅助调试代码

1. 什么是辅助调试代码

辅助进行代码调试的代码，帮助快速检测错误，应该尽早的引入辅助调试代码。

2. 移除辅助调试代码的方法

辅助调试代码会导致软件体积变大并且速度变慢。

可以使用 **ant** 和 **make** 等版本控制工具和 **make** 工具。

可以使用内置的预处理器：用编译器开关来包含或排除调试用的代码。

（九）产品中应当保留多少防御式编程代码

开发阶段：显示出的错误越多越好，引入很多防御式代码

发布阶段：希望错误尽可能的偃旗息鼓，不要影响代码执行

保留检查重要错误的代码

删除检查细微错误的代码

去掉导致系统硬性崩溃的代码

保留可以让程序稳妥地崩溃的代码

为技术支持人员记录错误信息

（十）如何对待防御式编程

适度进行防御式编程：程序臃肿、复杂度提高、缺陷引入。

应该采用防御的姿态，考虑好防御的地方和防御式编程的优先级。

六、表驱动法

（一）什么是表驱动法

表驱动法是一种编程模式，从表里面查找信息而不使用逻辑语句(if 和 else)

表驱动法适用于复杂逻辑，将复杂逻辑从代码中独立出来方便单独维护

（二）表驱动法实现与细节

表驱动法的表中主要存放数据，在一些特殊情况下也存放动作。

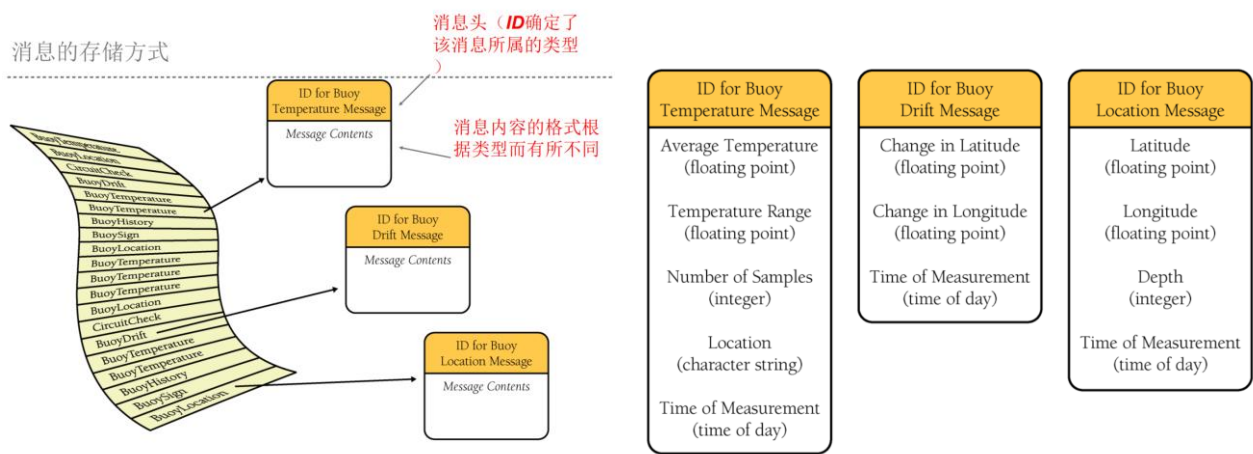
1. 直接访问(Direct Access)表

直接通过索引值可以从表中找到对应的条目。

a) 实例一：获取每月天数：使用天数数组

b) 实例二：灵活的消息格式

程序目标是打印文件中的消息，文件中存储约 500 条信息，每种文件存储了约 20 中不同的消息，每个消息包含消息表示符 ID 和若干字段，并且存储方式的消息头存储也不是固定不变的。



基于逻辑的方法：

顺序读取，检查 ID 后调用子程序阅读、解释和打印消息。

子程序数量较多，并且有一个消息格式边了，就需要修改对应的子程序逻辑。

修改：动作存储到表中（入口地址）

面向对象的方法：

问题的逻辑可以隐藏在对象继承结构中。

表驱动法：

消息阅读子程序由一个循环组成。

使用表来描述各种信息的格式避免硬编码。

使用一个消息打印子程序替代了 19 种消息的打印子程序。

每次读取信息头对 ID 解码后进行对应处理，每次调用对应子程序处理。

```
// C++示例：定义消息数据类型
enum FieldType {
    FieldType_FloatingPoint,
    FieldType_Integer,
    FieldType_String,
    FieldType_TimeOfDay,
    FieldType_Boolean,
    FieldType_BitField,
    FieldType_Last = FieldType_BitField
};
```

示例：定义浮标温度消息的表记录项

```
Message Begin
  NumFields 5
  MessageName "Buoy Temperature Message"
  Field 1, FloatingPoint, "Average Temperature"
  Field 2, FloatingPoint, "Temperature Range"
  Field 3, Integer, "Number of Samples"
  Field 4, String, "Location"
  Field 5, TimeOfDay, "Time of Measurement"
Message End
```

有多少种类型的消息，就需要定义多少个表项

ID for Buoy Temperature Message
Average Temperature (floating point)
Temperature Range (floating point)
Number of Samples (integer)
Location (character string)
Time of Measurement (time of day)

1.2.6. 表驱动法代码

1. 表驱动法中最上层循环的伪代码：这部分的代码与前面的方法差别不大

```
While more messages to read
  Read a message header
  Decode the message ID from the message header
  Look up the message description in the message-description table
  Read the message fields and print them based on the message
description
End While
```

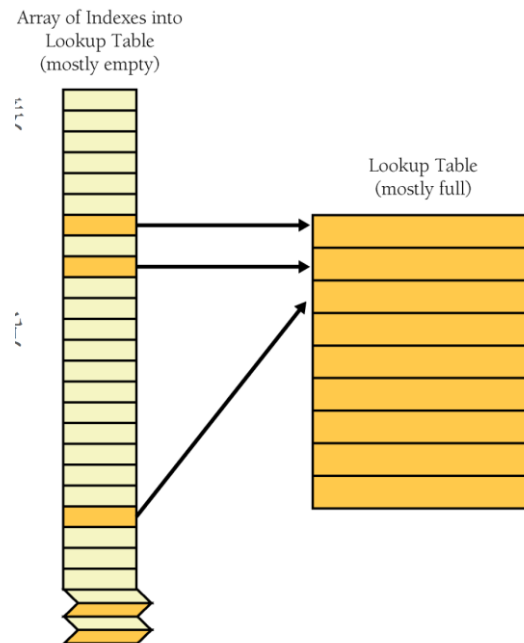
1.2.7. 消息打印子程序

```
While more fields to print
  Get the field type from the message description
  case ( field type )
    of ( floating point )
      read a floating-point value
      print the field label
      print the floating-point value
    of ( integer )
      read an integer value
      print the field label
      print the integer value
    of ( character string )
      read a character string
      print the field label
      print the character string
  ...
End Case
End While
```

2. 索引访问(Indexed Access)表

当无法直接从表中查询需要的条目时，先用基本类型的数据从索引表中找到对应的键值，然后根据键值查找出对应的主数据。

实例：现有一家商店，店内有约 100 种商品，每种商品有一个 4 位数字的物品编号，其范围为 0000-9999。如果使用直接访问表则需要生成一张 10000 条的访问表(除了 100 种商品记录外有大量空记录)。如果使用索引访问表，则需要生成一张 10000 条的索引表，然后其中对应的记录指向商品信息的数据表（记录位置）



优点：

- 如果主查询表中每条记录都很大，那么索引数组节省很多空间（索引记录只需要 2-4 字节）。
- 即使使用索引记录没有节省内存空间，操作位于索引中的记录也比操作主表中的记录更方便廉价。
- 编写到表中的数据相对于嵌入代码的数据更容易维护。

3. 阶梯访问(Stair-Step Access)表

不像索引表直接，但是比索引访问更节省空间。

基本思想：通过每项命中的阶梯层次确定其归属。

实例：成绩定档（A-F），使用区间表。

```
' set up data for grading table
Dim rangLimit() As Double = { 50.0, 65.0, 75.0, 90.0, 100.0 }
Dim grade() As String = { "F", "D", "C", "B", "A" }
maxGradeLevel = grade.Length - 1
...
' assign a grade to a student based on the student's score
gradeLevel = 0
studentGrade = "A"
While ( gradeLevel < maxGradeLevel )
    If ( studentScore < rangLimit( gradeLevel ) ) Then
        studentGrade = grade( gradeLevel )
    End If
    gradeLevel = gradeLevel + 1
Wend
```